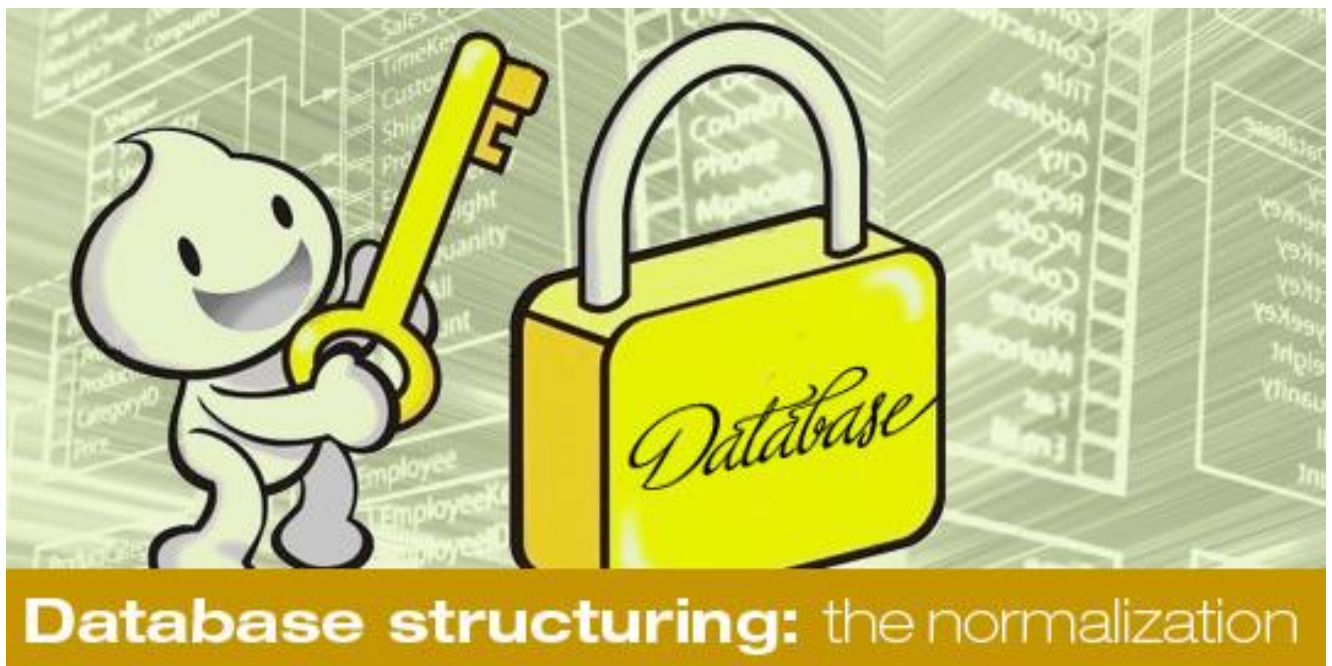


Note til Databaser

Mads Brix og Finn Nordbjerg

Nordjyllands Erhvervsakademi 2003

Revideret af KNOL/ALCL 2019



Uformelle og formelle kvalitetsmål for tabeldesign

I det følgende beskrives de krav, man må stille til tabeller i den relationelle model. Kravene beskrives både uformelt og formelt med udgangspunkt i normaliseringsteorien. Endvidere gives der et eksempel på et dårligt tabeldesign, som illustrerer hvilke problemer, der kan opstå. Teksten er skrevet med udgangspunkt i kapitel 14 fra den tidligere anvendte databasebog [Elmasri].

Forudsætninger

Der forudsættes kendskab til begreberne fra den relationelle model, dvs. tabeller, attributter, primær-, kandidat-, og fremmed-nøgler etc...

Uformelle regler for tabeldesign

Ved at følge nogle få, simple regler, som også tilgodeses ved at anvende ER-modeller eller objektmodeller (klassediagram), opnås normalt et godt tabeldesign. Disse regler er:

- Meningen med en tabel skal være let at forstå.
 - Man skal ikke sammenblande oplysninger om flere forskellige ting i den samme tabel.
- Undgå redundans
- Minimer antallet af null værdier

Redundans

Redundans optræder, når det samme faktum er registreret flere gange.

Eksempel: Det er redundant at registrere bynavnet hver gang postnummeret registreres.

Redundans er problematisk af to årsager, dels bruges der mere plads end det er nødvendigt, dels er der problemer med opdateringsanomalier.

Opdateringsanomalier

Et dårligt tabeldesign giver anledning til problemer, som under et betegnes opdateringsanomalier (update anomalies), men som opdeles i indsættelses-, sletnings- og modificeringsanomalier (insertion, deletion og modification anomalies). For at forklare disse anomalier tages der udgangspunkt i et eksempel.

Et eksempel

Som eksempel bruges følgende tabel, som er (en del af) en database til registrering af oplysninger i et firma. Tabellen *Employee_Project* repræsenterer oplysninger om de ansatte, hvilke projekter der er i gang, og hvem der har arbejdet i hvor mange timer på et projekt. Primærnøglen er sammensat af (*employee_no*, *project_no*).

Employee_Project

<u>employee_no</u>	<u>project_no</u>	projectname	hours	firstname	lastname	dateofbirth	education	monthly salary	department	department name
19	23	Tower block Living joy	103	Peter	Jensen	240165	Bricklayer	20000	1	New construction
19	45	Kindergarten Play Happy	10	Peter	Jensen	240165	Bricklayer	20000	1	New construction
2	23	Tower block Living joy	43	Anne	Andersen	230245	Secretary	19000	1	New construction
23	23	Tower block Living joy	250	Svend	Eriksen	111253	Carpenter	20000	2	Renovation
89	45	Kindergarten Play Happy	81	Arne	Jensen	071271	Carpenter	20000	1	New construction
145	88	Danmarksgade 23	93	Ingrid	Larsen	300369	Bricklayer	20000	2	Renovation

Anomalier

- *Modifikationsanomalier* er problemer med at huske at opdatere alle kopier af redundante data. I eksemplet vil man, for at ændre navnet på *department 1*, skulle ændre fire forekomster i tabellen.
- *Indsættelsesanomalier* er problemer i forbindelse med indsættelse af nye data. I eksemplet vil det være umuligt at oprette et nyt projekt uden at have mindst en ansat til at arbejde på det, da *employee_no* indgår i primærnøglen.
- *Sletningsanomalier* er problemer i forbindelse med sletning af data. I eksemplet vil sletning af et projekt, som er det eneste projekt en given medarbejder er tilknyttet, medføre at oplysninger om medarbejderen også slettes. F.eks. vil sletningen af projekt nummer 88 medføre at også medarbejderen med nummer 145 slettes fra databasen.

Formelle regler for tabeldesign - normalisering

Normalisering er en proces, hvorunder man undersøger om de tabeller, der arbejdes med, opfylder nogle regler, som er formuleret i de enkelte normalformer. Formålet er at sikre kvaliteten af tabeldesignet, bl.a. ved at fjerne redundans.

Der opereres formelt set med 6 normalformer, 1., 2., 3., BC, 4. og 5. normalform. Den underlige navngivning bunder i historiske årsager, BC normalform (Boyce-Codd normalform) blev formuleret efter introduktionen af 4. og 5. normalform. Normalform forkortes tit til NF, f.eks. benævnes Boyce-Codd normalform BCNF.

En redegørelse for 4. og 5. normalform falder uden for rammerne af denne tekst¹, en nærmere beskrivelse af 4. og 5. NF kan f.eks. findes i kapitel 21 i [Date].

¹ Påstand: et tabeldesign baseret på en gennemarbejdet EER-model vil aldrig opleve problemer med 4. og 5. normalform.

Funktionelle afhængigheder

Både 2., 3. og Boyce-Codd NF baserer sig på begrebet funktionelle afhængigheder (FA)². En funktionel afhængighed beskriver et forhold mellem to sæt af attributter, således at **værdien af det ene sæt af attributter (determinanten) bestemmer værdien af det andet sæt af attributter**. En funktionel afhængighed skrives normalt $X \rightarrow Y$, hvor X og Y er en eller flere attributter. F.eks. gælder den funktionelle afhængighed Postnummer $\rightarrow$ By. Det er funktionelle afhængigheder, som giver anledning til redundans, og derfor opstiller normalformerne regler for, hvilke funktionelle afhængigheder der må være i en tabel. Det er vigtigt at gøre sig klart at funktionelle afhængigheder afspejler forhold fra virkeligheden, så i sidste ende må man undersøge virkeligheden for at finde ud af, om en given regel (og der med en funktionel afhængighed) gælder.

Bemærk - at der i en tabel altid vil gælde funktionelle afhængigheder mellem primærnøglen (og evt. andre kandidatnøgler) og hver enkelt af de andre attributter.

Boyce-Codd normalform (BCNF)

BCNF siger at:

- **for alle funktionelle afhængigheder skal det gælde, at determinanten er en hel kandidatnøgle.**

Hvis en tabel er på BCNF er den også på 1., 2., og 3 NF og man kan derfor i princippet tage udgangspunkt i denne. Af hensyn til forståelsen (og den almene dannelse) er det dog nødvendigt at se på 1., 2. og 3 NF. Bemærk i øvrigt at BCNF forholder sig til, hvordan en god tabel skal se ud, mens 1., 2. og 3. NF forholder sig til, hvad der ikke må være i en god tabel.

En mere uformel formulering af BCNF er, at det, man gemmer oplysninger om i en tabel, skal afhænge **af nøglen, hele nøglen og intet andet end nøglen ('So help me Codd')**^{3,4}

Den Relationelle model

Normalformerne skal forstås i sammenhæng med den relationelle model, som er angivet som en forudsætning for læsningen af dette dokument. For at bruge normaliseringsteknikker på relationelle databaser, skal det sikres, at tabellerne er specificeret, så de er i overensstemmelse med den relationelle model, der er baseret på **matematiske mængder(sets)**⁵. Da den relationelle model specificerer en *relation* som en *mængde tupler*, **skal rækkerne i tabellerne være unikke**, da *sets* ikke tillader dublerede værdier. Det unikke sikres ved at identificere en **primær nøgle**. Det er vigtigt at forstå, at der ikke stilles nogen garanti for rækkefølgen af rækkerne, da *sets* er uordnede. Disse restriktioner på tabellerne danner således implicit grundlaget for begrebet normalformer, men de nævnes ikke eksplicit i nogen af dem.

1. normalform (1NF)

For at en tabel er på 1NF må den kun indeholde simple attributter, altså **ingen flerværdi eller sammensatte attributter**. 1NF er et levn fra gamle dage, idet en tabel i den relationelle model

² functional dependencies (FD)

³ **on the key, the whole key and nothing but the key ('So help me Codd')**

⁴ Det kan oplyses at Codd er faderen bag den relationelle datamodel, og læsere med hang til billige amerikanske retsalserier vil så måske finde denne bemærkning morsom.

⁵ Mathematical sets – **set** er en samling af distinkte objekter, der betragtes som et objekt i sig selv.

altid er på 1NF, 1NF er blevet en del af definitionen af den relationelle model. Den medtages mest af historiske årsager.⁶

2. normalform (2NF)

2NF forholder sig til partielle funktionelle afhængigheder. **En partiel funktionel afhængighed er en funktionel afhængighed, hvor det kun er en del af determinanten, der har betydning.** Hvis en funktionel afhængighed ikke er partiel, er den fuld. I en fuld funktionel afhængighed kan der ikke fjernes nogle attributter fra determinanten, uden at den funktionelle afhængighed ophæves.

En partiel afhængighed findes f.eks. i eksemplet, hvor det gælder at *projectname* ikke afhænger af hele nøglen (*employee_no*, *project_no*), men kun af *project_no*. Den funktionelle afhængighed (*employee_no*, *project_no*) → *projectname* er således en partiel afhængighed. Bemærk – at denne funktionelle afhængighed selvfølgelig også overtræder BCNF. Uformelt kan man sige at *projectname* kun er en egenskab ved *project_no*, og således ikke ved hele nøglen.

En tabel er på 2NF, når

- den er på 1NF
- alle ikke-nøgle attributter er fuldt funktionelt afhængige af en kandidatnøgle

3. normalform (3NF)

3NF forholder sig til transitive funktionelle afhængigheder. **En transitiv funktionel afhængighed er en funktionel afhængighed, der går via et andet sæt af attributter.**

I eksemplet ville der være en transitiv funktionel afhængighed mellem (*employee_no*, *project_no*) og *departmentname* gående via *department*

En tabel er på 3NF, når

- den er på 2NF
- ingen ikke-nøgle attribut er transitivt funktionelt afhængig af en kandidatnøgle.

Forskelle på 3NF og BCNF

Der er faktisk tabeller, som er på 3NF uden at være på BCNF. Dette skyldes primært den historisk betingede formulering “alle ikke-nøgle attributter...”, som siger, at hvis en attribut er en del af en nøgle, skal den ikke opfylde nogle krav. Hvis man i eksemplet antager, at flere projekter ikke kan have samme navn, vil der også være en kandidatnøgle bestående af (*employee_no*, *project_no*). Dermed er den funktionelle afhængighed *project_no* → *projectname* ikke længere et problem, idet *projectname* er en nøgle-attribut. Dog er der klare redundans-problemer, f.eks. er det faktum at *project_no* 23 hedder “Tower block Living joy” registreret tre gange.

Bemærk, at for at en tabel skal være på 3NF uden at være på BCNF, skal der være overlappende kandidatnøgler.

Hvordan normaliserer man?

For det første identificeres de funktionelle afhængigheder, som overtræder BCNF.

For hver af disse funktionelle afhængigheder (X→Y) splittes tabellen op, således at der oprettes en ny tabel bestående af alle attributter (både højre og venstre side, X og Y) af den funktionelle

⁶ Det skal bemærkes at det diskuteres at ændre definitionen af den relationelle model, således at der tillades flerværdi (og evt. sammensatte) attributter. Det vil betyde at tabeller ikke mere behøver være på 1NF, men giver nogle andre fordele, som det ligger uden for rammerne af denne tekst at redegøre for.

afhængighed, der giver problemet. I den gamle tabel efterlades determinanten (X), som nu bliver fremmednøgle, pegende over i den nye tabel.

Denne proces skal gentages for alle problematiske funktionelle afhængigheder, også for tabeller der oprettes som følge af normalisering af andre tabeller.

Kontrolleret redundans

Det skal nævnes at der i bestemte situationer kan indføres kontrolleret redundans for at optimere hastigheden på forespørgsler. Dette kan gøres i situationer hvor den funktionelle afhængighed afspejler en sammenhæng som typisk ikke ændres (f.eks. Postnummer->By), og hvor man vælger at bruge ekstra plads for at spare noget tid. Det skal understreges at kontrolleret redundans kun bør introduceres i undtagelsestilfælde, og først efter at man ved at normalisere helt igennem har fjernet al redundans.

Litteratur

[Elmasri]

Elmasri/Navathe

Fundamentals of Database Systems, 2nd ed.

The Benjamin/Cummings Publishing Company

ISBN 0-8053-1753-8

[Date]

C.J. Date

An introduction to Database Systems, Volume 1, 5th ed.

Addison-Wesley Publishing Company

ISBN 0-201-52878-9